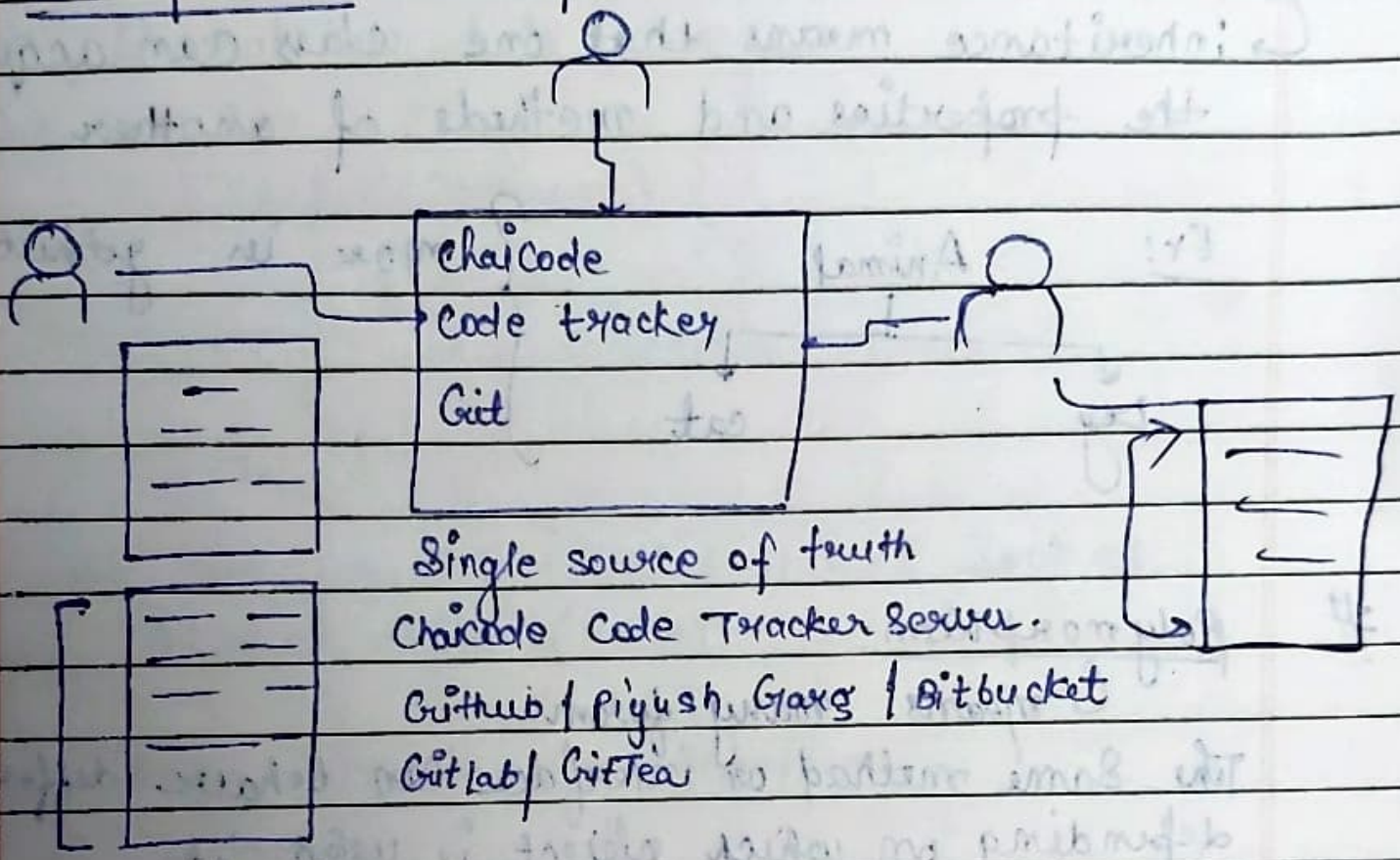
Git & Github

2/9/26



Single source of truth

ChaiCode Code Tracker Server.

Github / Piyush Garg / Bitbucket

Gitlab / GitTea

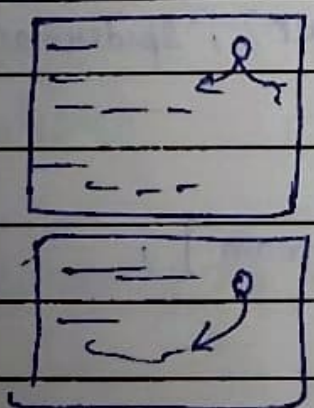
VCS (version Control System) - GIT

VCS & Remote - Remote - Github, Gitlab, GitTea, AWS Code commit, BitBucket

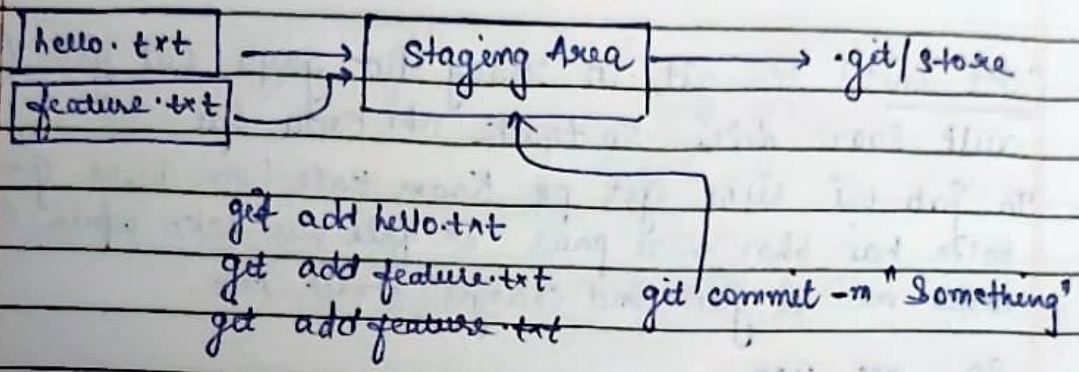
VCS (version Control System) → Git

↳ Server Host (Github)

.git/



chaicode tracker	git init .git/
ccct init chaicode	git add <file name>
ccct yaad-karo	git commit -m "maine aaj ye kiya"
ccct save-karo . Chaicode	



** Suppose there is a Js file

```
function add(a, b) {
    return a+b
}
add(2, 5)
```

If we want to add git to this file
First we need to write **git init**

1) **git init** : It initializes a new Git repository in my current folder.
It creates a hidden .git folder. This folder stores all the information Git needs to track your project.

2) **git Status**: It tells us the current state of your our working directory and staging area.

→ It helps us see:
1) which files are modified
2) which files are staged and ready for commit
3) which files are un-tracked
4) whether your working tree is clean
5) which branch you're currently on

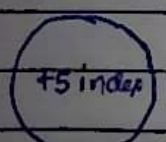
(Git init) se git to ready ho gaya. But git by default saare files ko track nhi karta hai.
To jab bhi hum git pe kaam karte hain hum git ko bolte hai bhai mere paas ek file hai isko apne staging area me le jao and changes track karo.
So, we use.

3) git add <filename> : Add the file to the git to be tracked.

4) git add . : It adds all the file in the directory recursively to git so that it can be tracked.

5) git Commit : It is used to save changes that you have staged into your Git repository's history.

git commit -m "Add login page"
↳ tells Git to create a commit
→ -m lets you provide message
"Add login page" ↔ describes what you change.



→ This is stored in .git directory which my local history.

add index file
2caca fcff49 ...
Author : I ...
Date : F ...

6) git diff: It is used to see exactly what has changed in you files since the last commit (or since staging).

For Example;

I had;

Hello World

and it changed to:

Hello Aditi

→ Running:

git diff

might show:

-Hello World

+Hello Aditi

7) git diff --Staged: Changes already staged.

8) git log: It is used to see the history of commits in Git repository.

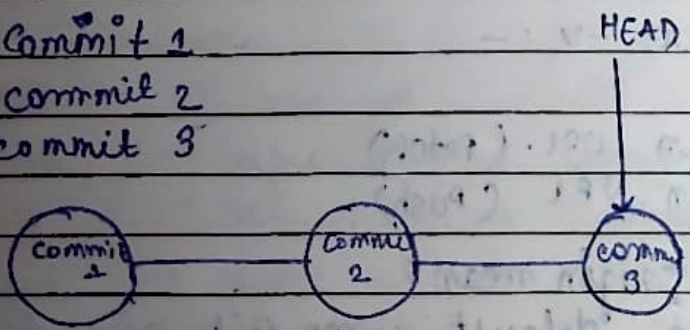
It shows information such as:

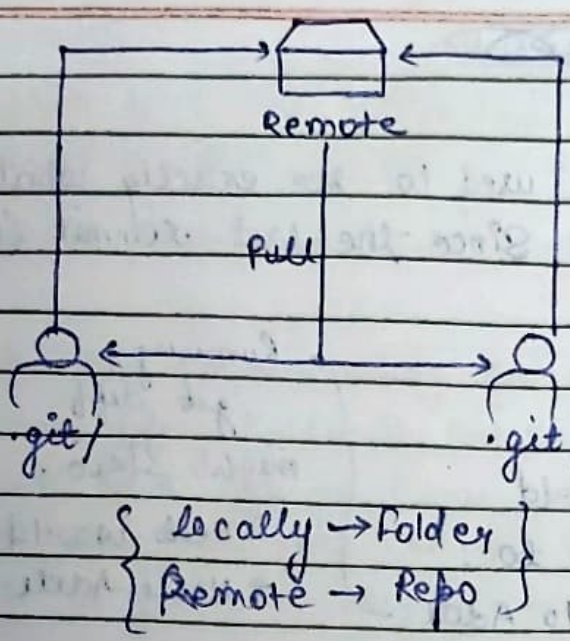
1) Commit ID
2) Author
3) Date
4) Commit message

Commit 1

commit 2

commit 3





9) git remove -v : This command shows the remote repositories comected to your local Git repository.

Connect

* How to ~~add~~ the local Git repository to the remote repositories ?

→ git remote add origin git@o <URL>

Now, when we write
→ git remote -v :-

Output: Origin URL. (Fetch)
origin URL (Push).

what does origin mean?
↳ Origin is the default name Git gives to your remote repository.
Think of it like:
your computer Github
my-project ⟶ Github
Origin

what are fetch and push?

(fetch) → Download changes from Github.

(push) → upload changes to Github.

10) git remote: To see only the remote name

11) git remote add origin <repository-Url>
↳ Add a remote.

12) change the remote URL:
↳ git remote set-url origin <new-url>

13) Remove a remote:
↳ git remote remove origin.

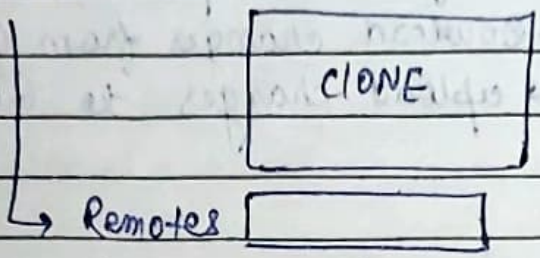
14) git push -u Origin main:

↳ This command is used to upload your local main branch to Github.

Breakdown:

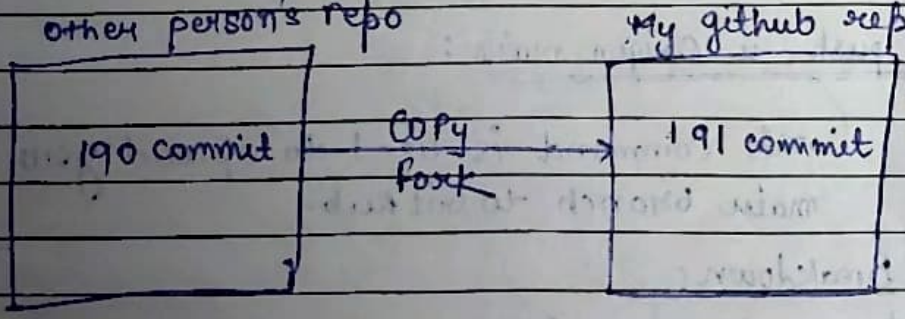
git → use git
push → upload / send commits
-u → set upstream tracking
Origin → name of your Github remote
main → branch you want to upload.

15) git clone <URL>



This command is used to copy/download an existing Github repository to your computer

16) git log --oneline:
↳ It displays the commit history in a compact, one-line-per-commit format
↳ Each line contains:
↳ Abbreviated hash (eg, a1b2c3d)
↳ commit message (Eg. fix login validation)



17 PULL REQUEST:

A pull request is a request to merge your code changes from one branch into another branch usually on Github.

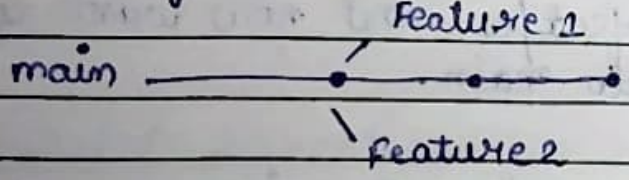
It is like "I have made some changes.

please review them and, if everything is good, add them to the main project".

Branches:

A branch in Git is basically a separate line of development where you can work on changes without ~~distributing~~ disturbing main code.

Think of it like:



The main branch contains the stable project while other branches are used to develop individual features.

(1) git checkout -b "feat/a":

↳ It means create a new branch named feat/a and immediately switch to it.

git checkout -b "feat/a"

↑ Create ↑ branch name

we can also write:

git switch -c feat/a

(2) git branch: Shows in which branch we are in.

(3) git merge <branchName>

↳ This command changes from another branch into the branch you are currently on.

For example:

Suppose I have

main

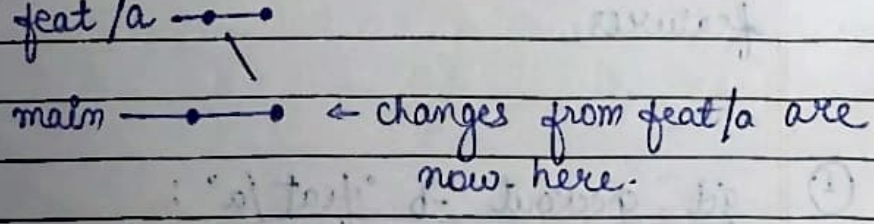
\ feat/a

* I worked on feat/a and now want to bring those changes into main.

Important: Step1: Switch to main

Step2: Merge feat/a

Now,



(4) git reset --hard <commit-id>

This command moves our current branch back to specific commit and discards changes after that commit.

Example:

Suppose our history is

A → B → C → D.

and I run:

git reset --hard B
My branch will become
A → B

C & D are no longer part of the current branch history.

what does --hard do?
→ It changes three things
1. HEAD → move to specified commit
2. Staging Area → reset to that commit
3. Working files → changed to match that commit

So, any uncommited changes in your files can be permanently lost

A useful distinction:

git reset --soft → move commit, keep changes staged
get reset → move commit, keep changes unstaged
git reset --hard → move commit, DELETE, changes.

(5) get merge --squash (feat/a).
branch name
↳ this is used to bring all the changes from another branch into your current branch as one single set of changes, without bringing that branch's individual commits.

Example:
main
A — B
feat/a
└ C → D — E

You are on main:
→ git switch main

Then,
git merge --squash feat/a

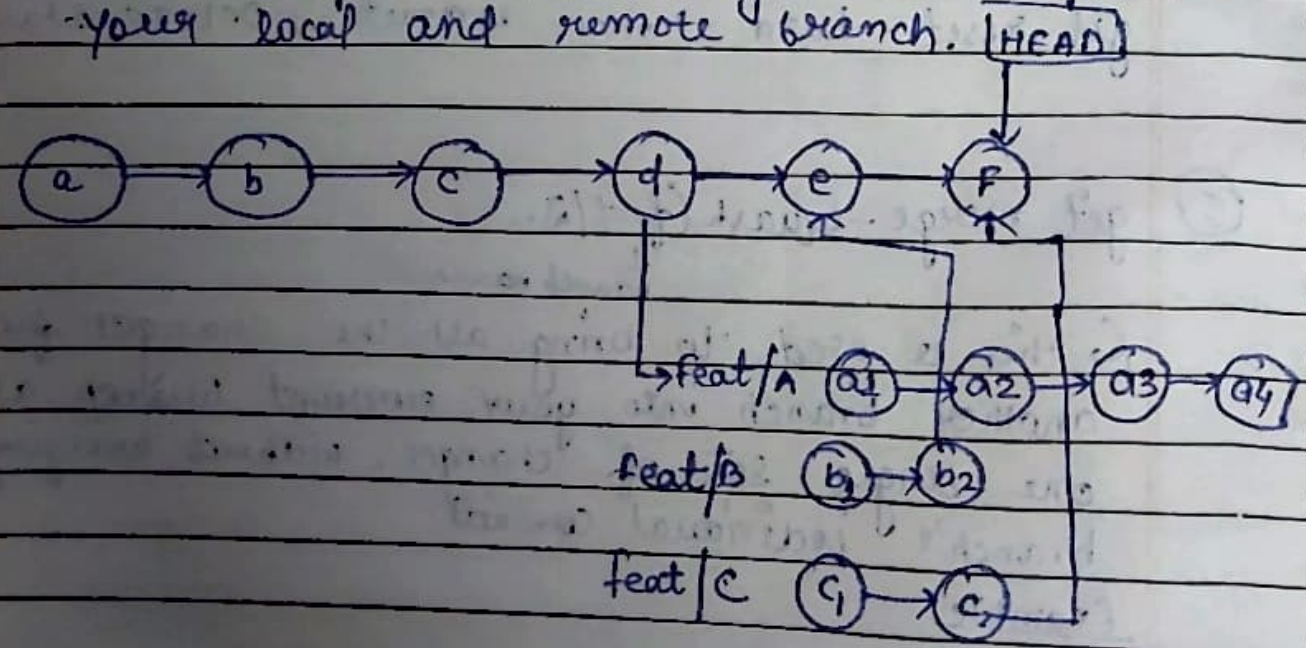
Git takes the combined changes from C, D and E and puts them into your working directory / staging area.
But it does not automatically creates a commit
So, you then do:

git commit -m 'Add feature A"
your history becomes
A - B - F

(6) git push --set-upstream origin feat/multiply.

→ This does two things
1. Pushes your local feat/a branch to Github (origin)
2. Sets the upstream / tracking relationship b/w your local and remote branch.

Imp:



what is the solu'n ?

→ Apni feature branch ko main k saath sync krne k liye rebase kro.
and main k. andr feature branch ko merge kro.

git rebase is used to move or replay your branch's commits on top of another branch.

⇒ In git, rebase is a command that integrates changes from one branch into another by moving or combining a sequence of commits to a new base commit. Unlike git merge, which creates a new "merge commit" to join divergent histories, git rebase rewrites the project history to a clean, linear sequence of commits, making it look as if the feature branch was created from the latest version of the target branch.